Lecture 4 · Machine-Level Programming I: Basics

10/1 (목) · 슬라이드 "Lecture 4 Machine-Level Basics" + 수업 녹취 정리 (Data Lab 2 부분은 Data Lab 페이지로 이동)

1 Intel x86 역사와 Architecture 흐름

CISC vs RISC

- **x86**: Intel 프로세서 계열의 ISA, laptop·desktop·server 시장을 지배
  - evolutionary design: 1978년 8086까지 backward compatible 유지 + 시간이 지나며 기능 추가
  - 모바일은 다른 생태계 → Qualcomm Snapdragon 등 ARM 계열
- 서버/데스크톱과 모바일은 trade-off가 다름: 서버는 가능한 한 빠르게 (performance), 모바일은 에너지 효율 (배터리)

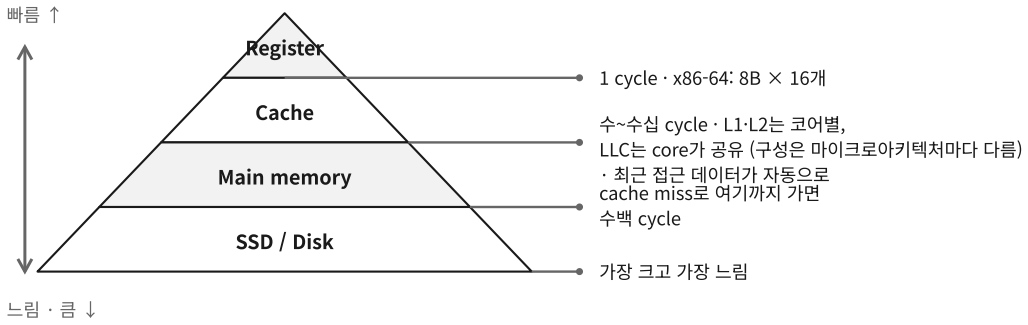
	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
명령어	형식이 제각각인 아주 많은 명령어, 가변 길이 1~15 byte	단순하고 형식이 고정된 명령어 (앞쪽 몇 bit가 opcode)
예	immediate를 메모리에 바로 저장하는 명령어 하나	immediate를 먼저 register에 넣고 → 메모리에 store, 2단계
특징	코드 줄 수는 적지만 하드웨어(decoder) 설계가 복잡. 실제 Linux 프로그램은 그중 일부만 사용	이해하기 쉬움

Transistor 수와 Multi-core 전환

- **트랜지스터 수**에 주목 → 공정 기술 발전으로 같은 면적에 더 많은 트랜지스터 → 더 복잡한 설계
- **Moore's law**: 트랜지스터 집적도가 주기적으로 두 배 (성능 두 배는 아님 — 과거 single-thread 향상은 clock 증가·설계 개선이 함께 뒷받침) → 예전에는 기다리기만 하면 single-thread 프로그램이 빨라졌음

- clock speed 상승의 한계 (내부 로직 복잡, 전력, 신호 간섭) → clock 올리기를 포기하고 core 수를 늘리는 방향으로 전환
- **backward compatibility**가 중요한 이유: 소프트웨어 전환은 하드웨어보다 훨씬 느림. 64-bit OS에서도 기존 32-bit application이 그대로 돌아야 함

Memory Hierarchy와 Register



메모리 계층: 빠를수록 작고, 클수록 느림. 자주 쓰는 데이터일수록 위쪽에 (슬라이드 그림 재구성)

- register: 가장 빠르고 가장 작은 "cache"로 볼 수 있음. 1 cycle 접근, x86-64 기준 8 byte짜리 16개

64-bit	32-bit	16-bit	8-bit
%rax %rbx %rcx %rdx %rsi %rdi %rbp %rsp	%eax %ebx %ecx %edx %esi %edi %ebp %esp	%ax %bx %cx %dx %si %di %bp %sp	%al %bl %cl %dl %sil %dil %bpl %spl
%r8 – %r15	%r8d – %r15d	%r8w – %r15w	%r8b – %r15b

- 32-bit register에 쓰면 상위 32-bit는 0으로 clear됨 · 16/8-bit 쓰기는 상위 비트 유지
- 함수 인자 전달: 첫 6개 정수·포인터 인자 = %rdi, %rsi, %rdx, %rcx, %r8, %r9 (순서대로) · 7번째부터는 stack · 반환값 = %rax

---

AMD와 x86-64의 탄생

- Intel의 64-bit 역사

  - 2001: IA32 → IA64 (Itanium)로 급진적 전환 시도 → 완전히 다른 구조, IA32는 legacy로만 실행, 성능 실망
  - 2003: **AMD가 진화적 해법 x86-64 (AMD64) 발표**. 32-bit register를 64-bit로 확장하고, 32-bit 프로그램은 하위 절반만 쓰면 되게 함 → 프로그램 변경 없이 호환
  - 2004: Intel이 EM64T 발표 → x86-64와 거의 동일
  - 현재는 저가형 제외 모든 x86이 x86-64 지원 (32-bit 모드 코드도 여전히 많음)
- **수업 범위: x86-64만 다룸** (`gcc hello.c` = 기본 64-bit, `gcc -m64`)

2 C, Assembly, Machine Code

ISA vs Microarchitecture

> **machine code**: 프로세서가 실행하는 byte 수준 프로그램 · **assembly code**: machine code의 텍스트 표현 (human-readable), 거의 1:1 대응.
> **ISA (Instruction Set Architecture)**: 올바른 assembly/machine code를 쓰기 위해 알아야 하는 프로세서 설계 부분 (명령어 집합 명세, register). 하드웨어 abstraction. 명령어가 무슨 의미이고 실행하면 어떤 효과가 있는지만 정의.
> **microarchitecture**: ISA의 실제 구현 (cache 크기, core 주파수, register가 칩의 어디에 있는지). 프로그래머에게 숨겨짐.

- ISA 예: x86, IA-32, x86-64는 x86 계열이며 CISC로 분류한다. Itanium(IA-64)은 x86 계열과 별개의 EPIC 기반 ISA이다. ARM과 RISC-V는 RISC 계열의 ISA이다.

Assembly 프로그래머가 보는 상태

Programmer-visible state(C에서는 안 보이던 것)

PC (program counter)
다음 실행할 명령어의 주소. IA32는 %eip, x86-64는 **%rip**

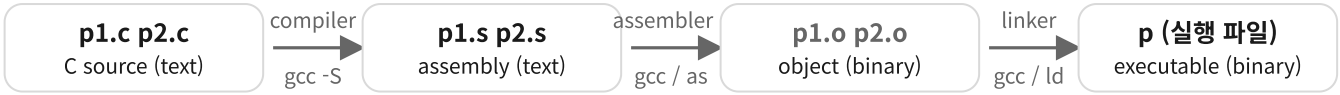
Register file
많이 쓰는 프로그램 데이터 저장

Condition codes
산술·논리·비교 명령어가 설정하는 상태 플래그이며, 조건 분기 명령어는 이 플래그를 읽어 if/else와 loop의 분기 여부를 결정한다

Memory
byte-addressable 배열: code·user data·(일부) OS data·stack 저장

기존 레지스터는 %rax와 %eax처럼 이름의 접두를 통해 64비트와 32비트를 구분한다. 확장 레지스터는 %r8, %r8d, %r8w, %r8b처럼 기본 이름에 d, w, b 접미를 붙여 각각 64비트, 32비트, 16비트, 8비트 폭을 구분한다. 흐름: 프로그램이 memory에 올라감 → CPU가 명령어 fetch → memory에서 data 읽기 → 연산 → write back.

C → Executable



`gcc -Og p1.c p2.c -o p` 한 번에 전부 수행. –Og: 기본적인 최적화만 (디버깅·읽기 쉬운 assembly)

- **assembler**: .s → .o, 각 명령어의 binary encoding. 거의 완성된 실행 코드 image, 단 다른 파일 코드와의 연결(linkage)이 빠져 있음
- **linker**: 파일 간 참조를 해결, static run-time library(malloc, printf 코드 등)와 결합. 일부 library는 dynamically linked → 프로그램 실행 시작 시 linking

sumstore 예제

```
long plus(long x, long y);
void sumstore(long x, long y, long *dest) {
    long t = plus(x, y);
    *dest = t;
    return;
}
```

```
sumstore:
    pushq %rbx        # rbx 값을 stack에 저장
    movq  %rdx, %rbx  # dest를 rbx로
    call  plus        # t = plus(x, y), 결과는 %rax
    movq  %rax, (%rbx)# *dest = t
    popq  %rbx        # rbx 되돌림
    ret
```

- `gcc -Og -S sum.c` → sum.s 생성. 같은 코드라도 기계·GCC 버전·설정에 따라 결과가 크게 다름
- 각 함수는 이름(label)으로 시작. 핵심: C의 함수 호출은 `call`, 대입 `*dest = t`는 register → memory 이동으로 바뀐다

Assembly의 특징

- data type
  - 정수: 1, 2, 4, 8 byte. **데이터 값과 주소(type 없는 pointer)를 구분하지 않음**
  - 부동소수점: 4, 8, 10 byte (별도 register 사용) · code: 명령어를 encoding한 byte 열
  - **aggregate type 없음** → array, struct는 memory에 연속 할당된 byte일 뿐 (주소를 직접 계산해서 접근)
- operation
  - register·memory 데이터의 산술 연산 (memory operand 산술은 x86 특유, RISC는 일반적으로 불가)
  - register ↔ memory 데이터 전송: load (memory → register), store (register → memory)
  - 제어 이동: procedure로의 무조건 jump, conditional branch (if/else 구현)

Object Code와 Disassembly

```
0000000000400595 <sumstore>:
  400595: 53                    push   %rbx
  400596: 48 89 d3              mov    %rdx,%rbx
  400599: e8 f2 ff ff ff        callq  400590 <plus>
  40059e: 48 89 03              mov    %rax,(%rbx)
  4005a1: 5b                    pop    %rbx
  4005a2: c3                    retq
```

- sumstore의 object code: 주소 0x400595부터 총 14 byte, 명령어 하나는 1, 3, 5 byte. hex 나열만 봐서는 알기 어려움 → disassembler 필요
- `*dest = t;` → `movq %rax, (%rbx)` → 0x40059e: `48 89 03`. t는 register %rax, dest는 register %rbx, *dest는 memory M[%rbx]. 괄호 ( ) = register 값을 주소로 보고 그 memory 위치에 접근
- disassembler: `objdump -d sum`: 명령어 bit pattern을 분석해 대략적인 assembly 재구성. source 없이 object만 있을 때 분석(reverse engineering) 수단. `gdb sum` → `disassemble sumstore`, `x/14xb sumstore`

손계산 1 · sumstore byte 수와 call 대상 주소

- 각 명령어 길이: 53(1) · 48 89 d3(3) · e8 f2 ff ff ff(5) · 48 89 03(3) · 5b(1) · c3(1) → 합계 $1+3+5+3+1+1=14$ byte ✓
- 각 주소 = 이전 주소 + 이전 명령어 길이 (예: 0x400596 + 3 = 0x400599)
- call의 피연산자 `f2 ff ff ff`는 little-endian 32-bit 상대 offset = −14. 기준은 다음 명령어 주소 0x40059e: $0x40059e + (-14) = 0x400590$ → <plus> 주소와 일치 (보충)

x86-64 인코딩·디코딩 — Objdump Reading

```
지원 범위(Coverage)
• pushq/popq %rbp, retq, nop, leaveq
• movq %rsp,%rbp / movq %rbp,%rsp
• movq 기본 8개 레지스터 ↔ 메모리, ModRM/SIB/disp
• subq $imm8,%rsp
• movq $signed-imm32,%r64 (C7 /0)
• movabsq $imm64,%r64 (B8+r)
• call rel32
```

부분집합 제한: REX.W=0x48 고정. rax/rcx/rdx/rbx/rsp/rbp/rsi/rdi만 지원합니다.
R8–R15, REX.R/X/B, 주소 크기 override, segment prefix, 일반 x86-64 ISA 전체는 지원하지 않습니다.

인코드(Encode) 디코드(Decode)

명령어 movq %rsp,%rbp

인코딩 / 디코딩 실행

기계어 바이트 — Instruction Bytes

48 89 E5

필드	바이트	의미
REX.W	48	W=1, R=X=B=0
opcode 89	89	r64 → r/m64
ModRM	E5	mod=11, reg/rm은 레지스터

디스어셈블(Disassembly)
movq %rsp,%rbp

Hex bytes
48 89 E5

Round-trip 검증: 통과
인코드 → 디코드 결과가 의미상 동일합니다.

objdump 스타일
00000000: 48 89 e5   movq %rsp,%rbp

Bomb Lab 포인트: 주소 → 바이트열 → 어셈블리 순서로 읽습니다.

3 Assembly 기본 연산: mov, Addressing Mode

Operand의 세 종류와 movq

Operand	형태	의미	예
Immediate (즉시값)	`$Imm`	상수 값, C의 상수처럼 $로 시작 (인코딩 크기는 명령어마다 다름 — 대부분 imm ≤ 32-bit sign-extend, movabsq는 64-bit 상수 가능)	`$0x400` , `$-533` (C 정수와 같이 2's complement로 encoding)
Register	`%rax` …	register 16개 중 하나, 현재 값을 사용	%rax, %r13
Memory	`(%rax)` 등	해당 주소에서 시작하는 8개 연속 byte (quadword)	`(%rax)` = %rax가 가리키는 memory

- %rsp도 x86-64의 16개 범용 레지스터 중 하나이며 일반 명령어의 소스·목적지나 메모리 주소의 베이스로 사용할 수 있다. 다만 관례상 스택 포인터로 사용되며, SIB 주소 지정에서는 인덱스 레지스터로 사용할 수 없다.
- `movq src, dest` : src → dest로 "8 byte(quadword) 복사" (move가 아니라 copy)
  - Reg↔Reg: `movq %rax, %rdx` ↔ `rdx = rax` (C 대입문)
  - Imm→Reg: `movq $0x4, %rax` ↔ `temp = 0x4`
  - Imm→Mem: `movq $-147, (%rax)` ↔ `*p = -147`
  - Reg→Mem (store): `movq %rax, (%rdx)` ↔ `*p = temp`
  - Mem→Reg (load): `movq (%rax), %rdx` ↔ `temp = *p`
- **두 operand가 동시에 memory일 수 없음**. memory → memory 이동은 명령어 하나로 불가능, 반드시 register를 거쳐 두 단계. destination에 immediate도 불가.

손계산 2 : swap 추적

```
void swap(long *xp, long *yp) {
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
```

```
    *yp = t0;
}
```

```
swap:
    movq (%rdi), %rax  # t0 = *xp
    movq (%rsi), %rdx  # t1 = *yp
    movq %rdx, (%rdi)  # *xp = t1
    movq %rax, (%rsi)  # *yp = t0
    ret
```

- 초기: %rdi → &a (0x120), %rsi → &b (0x118), memory: a = 0x232, b = 0x873
- ① movq (%rdi), %rax → %rax = 0x232 / ② movq (%rsi), %rdx → %rdx = 0x873
- ③ movq %rdx, (%rdi) → a = 0x873 / ④ movq %rax, (%rsi) → b = 0x232 → 교환 완료
- 핵심: ③ 직후에는 두 칸이 모두 0x873 → 원래 값 0x232가 %rax에 보관돼 있어 ④에서 복구 가능. memory 두 값을 직접 맞바꾸는 명령어가 없으므로 register 두 개를 임시 저장소로 사용 (load 2번 + store 2번).

Addressing Mode의 일반형 $D(R_b, R_i, S)$

$$D(R_b, R_i, S) \;=\; \mathrm{Mem}[\,\mathrm{Reg}[R_b] + S \cdot \mathrm{Reg}[R_i] + D\,]$$

- D: 상수 displacement (offset), disp8/disp32로 인코딩 (disp32는 부호 확장) · R_b: base register (16개 정수 register 아무거나) · R_i: index register (**%rsp 제외**) · S: scale ∈ {1, 2, 4, 8}
- x86-64의 SIB 인코딩은 scale을 1, 2, 4, 8 중 하나로 지정하도록 정의한다. 이 값들은 흔히 사용하는 배열 원소의 크기와 일치하여 주소 계산에 편리하다. 예를 들어 Linux LP64 환경에서는 char가 1 byte, short가 2 byte, int가 4 byte, long과 포인터가 8 byte이다. 자료형 크기는 허용 scale 값의 원인이 아니라 이 값들이 유용한 이유이다.
- R_b = 배열 시작 주소, R_i = 인덱스, S = 원소 크기 → $\&a[i] = a + S \cdot i$ — 바이트 주소 계산 기준 (C 포인터 산술에서는 `&a[i] == a + i`, 컴파일러가 S = sizeof(a[0])를 자동 반영). D까지 더하면 struct 배열의 특정 field 접근도 한 번에 표현
- 특수 경우 (생략된 필드는 0으로 취급, S 생략 시 1):
  - Normal `(R)` : $\mathrm{Mem}[\mathrm{Reg}[R]]$, C의 pointer dereference `*p` 와 유사
  - Displacement `D(R)` : $\mathrm{Mem}[\mathrm{Reg}[R] + D]$, object의 시작 + 상수 field 위치

- `(Rb,Ri)` : $\mathrm{Mem}[R_b + R_i]$ · `D(Rb,Ri)` : $\mathrm{Mem}[R_b + R_i + D]$ · `(Rb,Ri,S)` : $\mathrm{Mem}[R_b + S \cdot R_i]$ · `D(,Ri,S)` : base 없음 → $\mathrm{Mem}[S \cdot R_i + D]$

손계산 3 · 주소 계산 (%rdx = 0xf000, %rcx = 0x0100)

- `0x8(%rdx)` : 0xf000 + 0x8 → **0xf008**
- `(%rdx,%rcx)` : 0xf000 + 0x100 → **0xf100**
- `(%rdx,%rcx,4)` : 0xf000 + 4×0x100 → **0xf400**
- `0x80(,%rdx,2)` : base 생략(=0) → 2×0xf000 + 0x80 = 0x1e000 + 0x80 → **0x1e080**
- 주의: 이 값들은 "주소"까지만. mov에 쓰이면 그 주소의 memory를 실제로 읽는다.

유효주소 계산기 — Addressing Modes

%rbx **0x040** %rcx **0x010** Scale 4

Displacement 8 **8 (0x0000000000000008)**

leaq — 주소 계산 | movq — 메모리 읽기

```
피연산자(operand): 8(%rbx,%rcx,4)

EA = Imm + R[%rbx] + R[%rcx] × S
   = 8 + 64 + 16 × 4
   = 136

64비트 EA = 0x0000000000000088

leaq 8(%rbx,%rcx,4), %rax
%rax ← 0x0000000000000088
메모리 참조 없음(no dereference).
```

메모리 — Little-endian

시작 주소	8바이트 (낮은 주소 → 높은 주소)	64비트 값
0x0000000000000078	78 00 66 55 44 33 22 11	0x1122334455660078
0x0000000000000080	80 00 66 55 44 33 22 11	0x1122334455660080

시작 주소	8바이트 (낮은 주소 → 높은 주소)	64비트 값
0x0000000000000088	**88 00 66 55 44 33 22 11**	0x1122334455660088
0x0000000000000090	90 00 66 55 44 33 22 11	0x1122334455660090
0x0000000000000098	98 00 66 55 44 33 22 11	0x1122334455660098

주소 지정 형태 — 행을 클릭하면 계산기에 적용

피연산자	계산	특징
(%rbx)	R[%rbx]	Base register
(%rbx,%rcx)	R[%rbx] + R[%rcx]	Base + index
16(%rbx)	16 + R[%rbx]	Base + displacement
16(%rbx,%rcx,4)	16 + R[%rbx] + 4×R[%rcx]	Base + scaled index
16(,%rcx,4)	16 + 4×R[%rcx]	SIB, base 없음
16	절대주소 16	Displacement-only. 명령 형식에 따라 인코딩 제약이 있습니다.
16(%rip)	다음 RIP + 16	RIP-relative. 절대주소와 다릅니다.

> leaq는 계산된 주소를 저장하고, movq는 그 주소에서 8바이트를 읽습니다.
> 표의 행을 클릭하면 해당 addressing form이 적용됩니다.
> Absolute displacement-only와 RIP-relative는 서로 다릅니다.
> RIP-relative의 기준은 현재 명령의 시작 주소가 아닌 다음 RIP입니다.
> 교육용 모델의 계산은 64비트 모듈러 주소 연산입니다.

4 lea(Load Effective Address)와 산술 연산

leaq : 주소만 계산하는 명령어

- `leaq src, dst` : src(address mode 표현식)가 나타내는 **주소 자체**를 dst에 저장. memory에 접근하지 않음

```
mov 6(%rbx,%rdi,8), %ax  ↔ ax = *(rbx + rdi*8 + 6)   … memory 읽기
add 6(%rbx,%rdi,8), %ax  ↔ ax += *(rbx + rdi*8 + 6)  … memory 읽기
```

```
xor %ax, 6(%rbx,%rdi,8)  ↔ *(rbx + rdi*8 + 6) ^= ax  … memory 읽기+쓰기
lea 6(%rbx,%rdi,8), %rax ↔ rax = rbx + rdi*8 + 6     … memory 접근 없음
```

- 대부분의 명령어에서 memory operand는 memory에 접근 → **LEA만 예외**
- 쓰는 이유:
  - CPU 설계자의 의도: pointer 계산 (예: 배열 원소 하나의 주소를 다른 함수에 넘길 때 `p = &x[i]` → `lea (%rbx,%rdi,8), %rax` ↔ rax = &rbx[rdi])
  - compiler는 일반 산술에도 즐겨 사용: $x + k \cdot y + d$ ($k \in \{1, 2, 4, 8\}$) 꼴을 명령어 하나로: `lea (%rbx,%rbx,2), %rax` ↔ rax = 3·rbx
  - LEA는 명시적 operand가 두 개인 명령어로, 첫 번째 operand인 주소식의 유효주소를 두 번째 operand인 레지스터에 저장한다. 주소식의 base, index, scale, displacement를 이용해 일반 산술 연산에도 활용할 수 있으며, condition codes는 변경하지 않는다.

손계산 4 · m12 (x × 12)

```
long m12(long x) { return x * 12; }
```

```
leaq (%rdi,%rdi,2), %rax  # t ← x + x*2 = 3x
salq $2, %rax             # return t << 2 = 4t
```

- `leaq`: $\text{rax} = x + 2x = 3x$ · `salq $2`: $\text{rax} = 3x \times 4 = 12x$
- 곱셈 명령어 없이 lea + shift로 곱하기 12 구현 ($12 = 3 \times 4$)

2-Operand 산술 명령어 (src, dst 순서 주의)

명령어	동작	비고
`addq`	dst = dst + src	
`subq`	dst = dst − src	**순서 주의: dst에서 src를 뺌**
`imulq`	dst = dst × src	
`salq`	dst = dst << src	shlq와 같음
`sarq`	dst = dst >> src (arithmetic)	sign bit 복제 → 음수 부호 유지

명령어	동작	비고
`shrq`	dst = dst >> src (logical)	왼쪽을 0으로 채움
`xorq` / `andq` / `orq`	dst = dst ^ / & / | src	

- 1-Operand: `incq` (dst+1), `decq` (dst−1), `negq` (−dst), `notq` (~dst). inc/dec는 loop counter 증감에 유용
- **add/sub는 signed/unsigned 구분 없음**. two's complement 덧셈·뺄셈은 bit 수준 동작이 동일 (Data Lab의 TAdd = UAdd와 같은 이야기)
- left shift는 부호 무관하게 오른쪽에 0 채움. right shift는 프로그래머가 선택: sarq(부호 복제) vs shrq(0 채움)

명령어 Suffix

- 대부분의 x86 명령어는 suffix를 붙이거나 생략한 두 형태 모두 표기 가능: `imul %rcx, %rax` = `imulq %rcx, %rax`
- suffix = 연산 크기: **b = byte (1), w = short (2), l = int (4), q = long (8)**. suffix가 있으면 register 이름과 크기를 맞춤
- compiler 출력(`gcc -S`)은 보통 suffix 있음 · disassembly(`objdump -d` , gdb disas)는 보통 생략 · Intel 매뉴얼은 항상 생략

손계산 5 · arith 추적

```
long arith(long x, long y, long z) {
    long t1 = x + y;   long t2 = z + t1;   long t3 = x + 4;
    long t4 = y * 48;  long t5 = t3 + t4;  return t2 * t5;
}
```

```
arith:
    leaq (%rdi,%rsi), %rax   # t1 = x + y
    addq %rdx, %rax          # t2 = z + t1
    leaq (%rsi,%rsi,2), %rdx # 3y
    salq $4, %rdx            # t4 = 3y << 4 = 48y
    leaq 4(%rdi,%rdx), %rcx  # t5 = x + t4 + 4
    imulq %rcx, %rax         # rval = t2 * t5
    ret
```

입력: %rdi = x, %rsi = y, %rdx = z(나중에 t4로 재사용), %rax = t1·t2·rval, %rcx = t5.

$x = 1,\ y = 2,\ z = 3$으로 추적 (시작: %rax=?, %rdx=3, %rcx=?)

leaq (%rdi,%rsi), %rax → %rax = 1 + 2 = 3 · %rdx = 3 · %rcx = ?

addq %rdx, %rax → %rax = 3 + 3 = 6 · %rdx = 3 · %rcx = ?

leaq (%rsi,%rsi,2), %rdx → %rax = 6 · %rdx = 2 + 2×2 = 6 · %rcx = ?

salq $4, %rdx → %rax = 6 · %rdx = 6 × 16 = 96 · %rcx = ?

leaq 4(%rdi,%rdx), %rcx → %rax = 6 · %rdx = 96 · %rcx = 1 + 96 + 4 = 101

imulq %rcx, %rax → %rax = 6 × 101 = 606 · %rdx = 96 · %rcx = 101

결과

반환값 = 606. C로 직접 계산해도 $t_5 = (1 + 4) + 96 = 101$, $t_2 = 3 + 3 = 6$, $6 \times 101 = 606$으로 일치

---

① 인자 3개가 %rdi %rsi %rdx로 전달됨 · ② y×48을 $3y \times 16$으로 분해 → lea(3y) + shift(×16) · ③ t3 = x + 4는 별도 명령어 없이 lea의 displacement 4로 흡수 · ④ C에는 곱셈이 2번이지만 assembly에는 imulq 1번뿐 · ⑤ C 문장 순서와 assembly 순서가 꼭 같지 않음 (compiler가 재배치)

5 시험 포인트

- ISA = 프로그래머가 알아야 하는 명세(명령어, register), microarchitecture = 구현(cache 크기, 주파수)
- CISC(x86): 가변 길이·여러 명령어
- RISC(ARM, RISC-V): 고정 형식·단순 명령어
- clock 상승 한계 → multi-core로 전환, backward compatibility가 x86 진화의 핵심
- x86-64는 AMD가 먼저 (AMD64), Intel의 IA64(Itanium)는 실패 → EM64T
- C → (compiler) .s → (assembler) .o → (linker + library) 실행 파일
- assembly에는 array/struct 같은 aggregate type이 없음 → 연속된 byte + 주소 계산
- programmer-visible state: PC(%rip), register, condition codes, memory
- %rsp는 stack pointer로 특수 용도, %rbp는 64-bit에서 general purpose
- 인자 순서: %rdi, %rsi, %rdx, 반환값 %rax
- mov operand: Imm/Reg/Mem 중 dest에 Imm 불가, memory→memory 한 명령어로 불가
- $D(R_b, R_i, S) = \mathrm{Mem}[R_b + S \cdot R_i + D]$, $S \in \{1, 2, 4, 8\}$: SIB 인코딩에서 허용하는 scale 값이며, 자료형 크기와 일치할 때 배열 원소의 주소 계산에 유용하다. R_i에 %rsp 불가
- lea는 주소만 계산하고 memory에 접근하지 않음, condition code 불변, $x + k \cdot y + d$ 산술에 활용
- AT&T 문법은 src, dst 순서 → subq src, dst는 dst − src
- sar = arithmetic(sign 복제), shr = logical, add/sub는 signed/unsigned 구분 없음
- suffix b/w/l/q = 1/2/4/8 byte

6 연습문제

문제 1. %rax = 0x100, %rcx = 0x3 일 때 다음 표현식이 나타내는 주소를 구하라.
① 4(%rax) ② (%rax,%rcx,8) ③ 0x10(%rax,%rcx,8) ④ (,%rcx,4)

정답 보기

① 0x100 + 4 = 0x104
② 0x100 + 8 × 3 = 0x100 + 0x18 = 0x118
③ 0x118 + 0x10 = 0x128
④ base 없음 → 4 × 3 = 0xc

결과 **① 0x104 ② 0x118 ③ 0x128 ④ 0xc**

문제 2. 다음 중 x86-64에서 유효하지 않은 명령어를 고르고 이유를 써라.
① movq $5, %rax ② movq (%rax), (%rbx) ③ movq %rax, $5 ④ movq %rdx, 8(%rsp) ⑤ movq (%rax,%rsp,4), %rbx

정답 보기

② memory → memory 이동은 명령어 하나로 불가 → register를 거쳐 두 번에 나눠야 함
③ destination이 immediate일 수 없음
⑤ index register R_i로 %rsp는 사용 불가

결과 **유효하지 않은 것: ② ③ ⑤ (① ④는 유효: Imm→Reg, Reg→Mem)**

문제 3. %rbx = 0x1000, %rdi = 3이고 주소 0x101e에 저장된 8 byte 값이 42일 때, 다음 두 명령어 실행 후 %rax 값을 각각 구하라.
① movq 6(%rbx,%rdi,8), %rax ② leaq 6(%rbx,%rdi,8), %rax

정답 보기

주소 = 0x1000 + 3 × 8 + 6 = 0x1000 + 0x18 + 0x6 = 0x101e
① mov는 그 주소의 memory를 읽음 → %rax = 42
② lea는 주소 자체만 저장 → %rax = 0x101e

결과 **① %rax = 42 ② %rax = 0x101e**

문제 4. 다음 assembly가 계산하는 C 함수를 쓰고, $x = 7$일 때 반환값을 구하라.

```
leaq (%rdi,%rdi,4), %rax
salq $2, %rax
ret
```

정답 보기

leaq : x + 4x = 5x
salq $2 : 5x × 4 = 20x
C 함수: `long f(long x) { return x * 20; }`
x = 7 → 5 × 7 = 35 → 35 × 4 = 140

결과 **f(x) = 20x, f = 140**

문제 5. 본문의 arith assembly를 $x = 2,\ y = 1,\ z = 3$으로 추적해 각 명령어 직후 %rax, %rdx, %rcx 값과 최종 반환값을 구하라.

정답 보기

leaq (%rdi,%rsi), %rax → %rax = 3

addq %rdx, %rax → %rax = 3 + 3 = 6

leaq (%rsi,%rsi,2), %rdx → %rdx = 1 + 2 = 3

salq $4, %rdx → %rdx = 48

leaq 4(%rdi,%rdx), %rcx → %rcx = 2 + 48 + 4 = 54

imulq %rcx, %rax → %rax = 6 × 54 = 324

결과

반환값 = 324. C 검산: $t_2 = 3 + (2 + 1) = 6$**,** $t_5 = (2 + 4) + 48 = 54$**,** $6 \times 54 = 324$ ✓

문제 6. %rax = −8 (64-bit)일 때 sarq $1, %rax와 shrq $1, %rax의 결과를 각각 구하라 (shr 결과는 hex로).

정답 보기

−8 = 0xFFFFFFFFFFFFFFF8

sarq : sign bit 복제 → 0xFFFFFFFFFFFFFFFC = −4

shrq : 왼쪽에 0 → 0x7FFFFFFFFFFFFFFC (큰 양수)

결과 **sar → −4, shr → 0x7FFFFFFFFFFFFFFC**